



The tech behind *re-forge*.

Not a model, not a graph runtime, not another coding assistant. re-forge is a hardened **operating procedure** on top of Claude Code's subagent primitives: a contract that forces a fleet of specialist agents to disagree, audit each other's sources, write evidence to disk, and carry lessons forward across sessions and projects. The binary is markdown plus shell. The test suite is a 685-check validation harness.

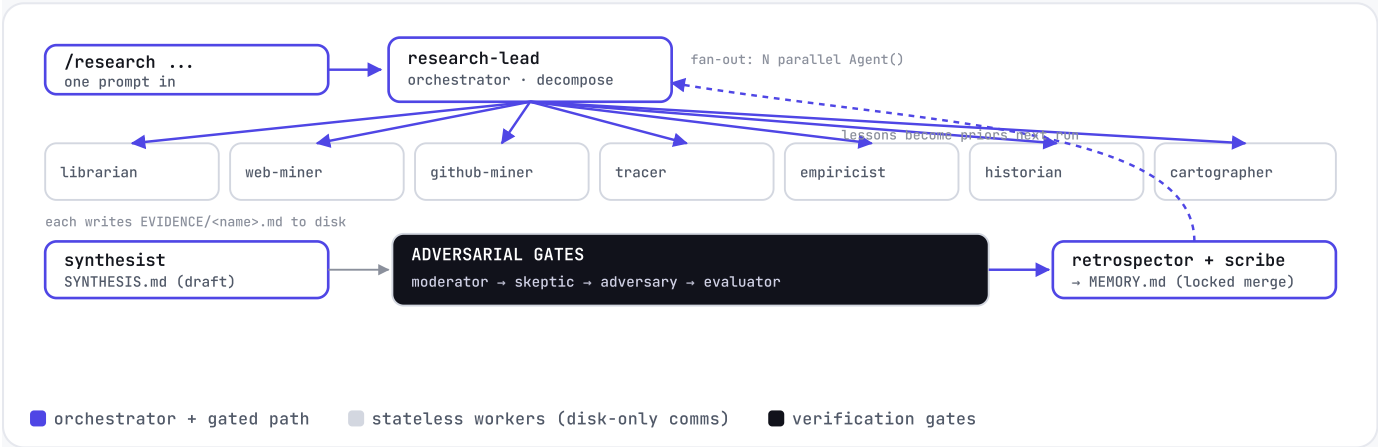
685 CHECKS
GREEN CI, EVERY PUSH

80+ AGENTS · 128 SKILLS
6 TEAMS + FORGE

2 PAPERS
ICML 2026 SCALE

01 Architecture: one prompt becomes coordinated specialist teams

The orchestrator (**research-lead**, **engineering-lead**, ...) is the single entry point. It does three things: **decompose** the question into disjoint sub-questions, **dispatch** specialists in parallel (one message, multiple `Agent()` calls, following Anthropic's 1 / 2-4 / 10+ workers scaling rule), and **sequence** the verification gates. Workers never talk to each other; they communicate only through evidence files on disk. That is deliberate: it removes the MAST conversation-failure mode where agents corrupt each other through unstructured chatter.



Two execution modes ship in every lead. **Main-thread mode**: the lead is the top thread and spawns specialists via the `Agent` tool with `run_in_background`. **Subagent mode**: when the lead is itself a dispatched subagent and cannot spawn its own, it runs the specialists' protocols inline, in sequence. Same procedure, degraded parallelism. This is what makes teams composable. Six specialist teams plus the **Forge** meta-agent (a gap → scout → draft → test → promote pipeline that authors new skills) total 80+ agent definition files. The Forge built the `hn-search` skill end-to-end in a smoke test: every session makes the next smarter in *capability*, not just memory.

02 Adversarial verification gates

Most multi-agent systems fail by adding agents without adding verification: more voices, same confident errors. re-forge imports the **MAST failure taxonomy** (NeurIPS 2025, 14 failure modes) and gives each relevant mode an owner. A SYNTHESIS cannot be marked high-confidence until it survives four gates in fixed order. The gates run *after* synthesis with a mandate to reopen, which is the structural cure for anchoring: anchoring is a single-pass disease, and this pipeline is multi-pass with an adversary that never saw the synthesist's framing.

GATE	ATTACKS	MAST	WHY IT WORKS
● Skeptic	the reasoning	FM-3.3	Hunts unsupported leaps, overgeneralization, single-source claims dressed as consensus. Forces each conclusion to re-derive from evidence, not from the first framing.
● Adversary	the sources	FM-3.3*	Treats the corpus as hostile: SEO farms, benchmark fraud, citation laundering, astroturfed stars. A different job from the skeptic: reasoning can be flawless on top of a fraudulent source.
● Moderator	contradictions	FM-2.5	When two workers disagree, runs a 3-round debate with five verdict types (A_WINS, B_WINS, COMPLEMENTARITY, REFRAME, DEFER), removing leader bias from conflict resolution.
● Evaluator	the output	FM-3.2	Scores a 5-dimension rubric (Anthropic's). A strict-dimension miss blocks PASS.

WHY THE SKEPTIC / ADVERSARY SPLIT IS LOAD-BEARING

In the first pilot, the **adversary caught a widely-starred memory project** publishing a 96.6% LongMemEval R@5 number that, on reading the benchmark file, actually measured a vanilla ChromaDB lookup, not the advertised architecture. The skeptic alone could not have caught it: the

reasoning around the number was internally coherent. Only an agent whose entire job is to read the source artifact adversarially surfaced it. That is the architectural argument for separating "attack the argument" from "attack the evidence."

03 The memory system: ACE plus a file-locked concurrent merge

Memory follows the **ACE pattern** (Agentic Context Engineering, Stanford): three roles, three phases.

- **Generator** (retrospector): at session close, extracts 3-7 durable lessons from evidence and verdicts.
- **Reflector**: applies a durability test, "would this still help in 3 months, on an unrelated question?" Session trivia is dropped; transferable patterns survive.
- **Curator** (scribe): merges survivors into persistent `MEMORY.md`, dedupes via grow-and-refine, marks stale entries.

Memory is **global** (`~/ .claude/agent-memory/<agent>/`) so lessons transfer across projects, while sessions are **per-project** so evidence stays isolated. The next lead consults its playbook *before acting*: lessons become priors on the next dispatch.

THE HARD PROBLEM

Multiple teams run in parallel and finish near-simultaneously, each appending to the same **MEMORY.md**. Naive append corrupts. The solution is a file-locked, atomic merge: the retrospector writes to `staging/<slug>.md`; the scribe runs the canonical merge below. Measured: **10 concurrent scribes, ~70ms total, zero lost writes, zero duplicates**.

```
flock -w 5 -x "$LOCK" timeout --signal=KILL --kill-after=1 30 bash -c '
set -e
TMP="$MEM.tmp.$$"
[ -f "$MEM" ] && cp "$MEM" "$TMP" || : > "$TMP"
for f in "$STAGING"/*.md; do
  cat "$f" >> "$TMP"
  mkdir -p "$STAGING/_merged"; mv "$f" "$STAGING/_merged/"
done
mv "$TMP" "$MEM"          # atomic rename: readers never see a half-merge
' || { echo "deferred merge: staging preserved" >&2; exit 0; }
```

- **flock -w 5 -x**: exclusive advisory lock with a 5s acquire timeout, so writers serialize instead of racing.
- **timeout --signal=KILL --kill-after=1 30**: required, not decoration. A bare `flock -c` leaks the lock on child-process inheritance when the parent is killed. Found empirically.
- **temp + mv**: `mv` is atomic on POSIX, so a concurrent reader sees the old file or the fully-merged new one, never a torn write. On lock timeout the merge **defers and exits 0**: staging is durable, the next scribe picks it up. No lost writes, no aborted session.

04 Evidence-as-contract: how auditability works

The integrity rule: **every specialist writes its evidence to disk, and only the named specialist writes its file**. A worker that ran but produced no `EVIDENCE/<name>.md` is treated as a worker that did not run. This "full-activation enforcement" makes shortcutting *detectable* rather than silent. On top sits **Eval-Driven Development** (binding on every team): no production code, test, or refactor is written until the criteria it must satisfy already exist on disk in `EXPECTED_EVALS.md`.

THE EDD CYCLE

1. **Define evals first** (per-dimension pass conditions: correctness, security, accessibility/perf, maintainability). 2. **Implement against the evals**, which are the spec. 3. **Verification loop**: the verifier runs each check fresh, no cached output, and pastes raw results into `EVIDENCE/verification.md`. 4. **Gate on evidence**: the evaluator may not record PASS until every criterion is met or covered by a documented exception.

THE PAYOFF: RECONSTRUCT THE VERDICT FROM FILES

Anyone can reconstruct **why** a session reached its verdict by reading files, not transcripts:

```
python3 ~/.claude/scripts/audit_evidence.py
<slug> --gate=synthesis --strict -v
```

It verifies every dispatched specialist has an evidence file, that EDD artifacts are complete, and that a PASS is backed by a reconciled checklist. An unrecorded gap is a defect; a recorded exception is legitimate evidence. The difference between "the agent says it's done" and "here is the on-disk proof."

05 Model and tool-agnostic by design: a procedure, not a model

re-forge owns no model and bets on no single assistant winning. It is a layer of **procedure** (decomposition, adversarial gating, evidence contracts, accumulated memory) expressed as markdown and shell. None of that is coupled to a specific model's weights. Today the procedure runs on Claude Code's primitives and agents declare Opus at max effort, with graceful fallback to a plan's strongest available model. But the contract is model-independent: when the underlying model changes, the procedure is unchanged and only the executor swaps. That is what lets the same operating procedure span Claude Code, Cursor, Codex, and ChatGPT. **re-forge wins regardless of which base model or assistant wins the market**, because it sits *above* all of them as the memory, skill, and audit layer.

06 Cross-tool monitor and live team-memory sync (roadmap, honest)

ROADMAP This is the product vision, not shipped traction. The shipped product today is for Claude Code. Here is how the cross-tool layer works technically, built strictly on primitives that already exist:

- **Always-on monitor across tools.** A lightweight, read-mostly local daemon observes session activity across Claude Code, Cursor, Codex, and ChatGPT (via session logs, extension hooks, local surfaces), detects recurring friction, and feeds the signal to the Evolution team and the Forge.
- **Forged personalized skills.** On a recurring gap, the Forge's gap → scout → draft → test → promote pipeline (already shipped) authors a personalized skill, driven by your actual usage.
- **Live shared team memory.** The same flock + timeout + atomic-rename merge that already gives safe parallel writes within Claude Code generalizes to multiple developers and tools writing into one lock-serialized store. Because the merge is already proven safe at 10 writers, the multi-developer case is an extension, not a new invention.

HONESTY BOUNDARY

The cross-tool monitor and multi-developer sync are **roadmap**. They are credible because the hard part, concurrency-safe merge and a working Forge pipeline, is already shipped and tested. **What is not yet shipped** is the cross-tool ingestion surface and the multi-tenant sync service on top of it.

07 Two ICML 2026 SCALE Workshop papers

Both were accepted to the SCALE Workshop at ICML 2026, and both had their research and engineering runs orchestrated *by re-forge teams*: the system used itself to produce research that fed back into its own prompts. The findings, **verifier-gated aggregation** and **tool-registry re-prompting**, were merged into the agents' prompts and memory. The papers are both output of the system and inputs to its next version.

● SCALE WORKSHOP · ICML 2026

Compute-Aware Mixture-of-Agents: Verifier-Gated Adaptive Aggregation under a Fixed Token Budget

Shine Gupta · S Akash

A mixture-of-agents scheme that gates aggregation through a **verifier** and adapts how much it aggregates under a **fixed token budget**, instead of aggregating many outputs linearly. The principled version of "don't run every gate at full width when the budget is tight."

● SCALE WORKSHOP · ICML 2026

Who Hallucinates Tools, How Often, and What Fixes It?

S Akash · Shine Gupta

An empirical study of **tool hallucination** (inventing a tool that does not exist, or misusing one that does): how often, and what fixes it. The fix fed back as **tool-registry re-prompting**: agents are re-grounded on the actual available tool set, cutting invented-tool calls.

Provenance: S Akash is a EuroLLVM 2026 speaker (IIT Patna), CERN GSoC, and a vLLM contributor. Paper links public at camera-ready.

08 The 685-check validation harness as rigor evidence

Because there is no application code, **the harness is the product's test suite**. It proves the procedure is internally consistent, installs cleanly on a fresh machine, and cannot silently rot. It runs green in CI on every push (685 checks), plus `markdownlint` across every markdown file.

- ✓ **Frontmatter.** Every agent/skill YAML is valid and the invariant `name : = dir/filename stem` holds across 80+ agents and 128 skills, so `/forge:gap` resolves correctly.
- ✓ **Shell.** Every entry point is `shellcheck`-clean and enforces `set -euo pipefail` where required. Hooks are exempted from `set -e` (an aborting hook can kill the host session), and that exemption is itself test-enforced.
- ✓ **Install smoke.** A fresh-HOME end-to-end install: `setup.sh` into a clean \$HOME, then `doctor.sh` must exit 0. Proof that "git clone + bash setup.sh" works off the author's machine.
- ✓ **Memory-MCP.** The optional SQLite + FTS5 memory server boot-tests.
- ✓ **Links.** No dead links across repo markdown; a slower `external` job checks live URLs over the network.
- ✓ **Catalog.** `docs/CATALOG.md` is generated from frontmatter and `drift-checked`: hand-edit or forget to regenerate, CI fails. The inventory cannot go stale.
- ✓ **Hooks.** The three registered hooks (Stop, PostToolUse, SessionStart) install and behave correctly.
- ✓ **Uninstall + prose counts.** `uninstall.sh` removes exactly what it installed, leaving user lessons and third-party skills untouched; headline claims are kept honest against the repo.

09 Technical Q&A: the eight hardest questions

Q1 Scalability: does this fall over as you add teams?

No. Workers are stateless w.r.t. each other and talk only through disk, so adding specialists adds breadth without $O(n^2)$ chatter. Cross-team parallelism is bounded by the host's concurrency cap and the merge, proven safe at 10 writers. The ceiling is the host runtime, not re-forge's design.

Q3 Security of on-disk evidence: tampering?

Evidence lives under the project tree and `~/claude/`, governed by OS file permissions, the same trust boundary as source. The guarantee is **detection**, not DRM: the audit script fails on any missing/incomplete file or an unbacked PASS. Merges are atomic, so a concurrent writer cannot corrupt the record mid-write.

Q5 What happens on a model change?

Almost nothing, by design. The procedure is markdown + shell, decoupled from weights. Agents declare a preferred model with graceful fallback; swap the model and the contract (gates, evidence, merge) is identical. This is the entire point of being a procedure, not a model.

Q7 Latency?

Two regimes. Main-thread mode fans workers out in parallel, so wall-clock is roughly the slowest worker plus the serial gate chain (a toy question is 2-3 min). Subagent mode degrades to serial: correct but slower. The merge adds **~70ms even at 10 writers**. Latency is dominated by investigation depth, which the user controls via the command.

Q2 Cost: adversarial gates sound expensive.

An explicit, bounded cost. A toy `/research` lands SYNTHESIS in 2-3 min under 50K tokens; a full investigation is 50K-500K. The alternative is a cheap, confident, wrong answer (the fraud the adversary caught). The Compute-Aware MoA paper is literally our research on spending a **fixed budget** optimally. Cost is a dial, not a fixed tax.

Q4 How does memory avoid context bloat?

Three mechanisms. The reflector's **durability test** drops trivia at generation (3-7 lessons/session). The curator **dedupes** via grow-and-refine and marks stale entries. **Hook A** routes long-tail overflow to topic files with pointers, keeping hot MEMORY.md small. It is a curated playbook, not an append-only log.

Q6 Data privacy of the cross-tool monitor (roadmap)?

Designed **local-first**: it observes session activity on the developer's machine, and the shared store is the team's own (Enterprise tier offers on-prem memory and SSO). Nothing requires shipping source or transcripts to a third party for the core loop. Stated honestly: a design commitment, not yet a shipped, audited control.

Q8 How are skills forged?

Five stages. **gap** diffs the capability set against a taxonomy (often seeded by a recurring memory lesson); **scout** checks the MCP Registry and marketplaces to reuse first; **draft** wraps the official skill-creator/mcp-builder; **test** runs a smoke case; **promote** installs it with helpful/harmful counters. Proven end-to-end on `hn-search`, then drift-checked by the same 685 harness.

